UNIVERSIDADE FEDERAL FLUMINENSE

JOÃO VICTOR DE SOUZA PIMENTEL CUNHA

SUPORTE A DESIGN RATIONALE EM ENGENHARIA DE SOFTWARE DIRIGIDA POR MODELOS USANDO A ABORDAGEM KUABA

Extensão da ferramenta Modelio para apoiar Modelos de Discussão com a Ontologia Kuaba

RIO DAS OSTRAS

2024

Lista de Figuras

1 Elementos do vocabulário da ontologia Kuaba. 3

2 Fluxo MDA 5

3 Ícones criados 9

4 Exemplo de Definição de Perfil e Estereótipos 10

5 Perfil Kuaba 11

6 Configuração de novos Comandos 12

7 Configuração de novos Diagramas 13

8 Configuração de novos elementos de paleta 14

9 Classe de criação e estilização de Links 15

10 Configuração do Wizard para o diagrama 15

11 Wizard de Criação 16

12 Relacionamento das classes para o Wizard de criação 16

13 Método construtor da classe DiagramBase 17

14 Método *actionPerformed* da classe *PIMDiagramWizardContributor* 18

15 Aba de Propriedades Personalizada 19

16 Hierarquia da Aba de Propriedades 19

17 Código das propriedades do elemento Questão 21

18 Código do método *actionPerformed* da classe *TransformCIMToPIM* 22

19 Código do método *processDecisions* da classe *TransformCIMToPIM* 23

20 Criação dos pacotes *Kuaba Package*, CIM e PIM 24

21 Criação de diagrama 25

22 Criação do elemento Questão 26

23 Criação dos elementos Ideia 27

24 Criação dos elementos Argumento 27

25 Criação dos elementos de Decisão 28

26 Execução da transformação 29

27 PIM após transformação 29

28 PIM finalizado pelo usuário 30

Sumário

1 Introdução **1**

2 Design Rationale **2**

2.1 Ontologia Kuaba 2

3 *Model-Driven Architecture* - MDA **4**

3.1 Modelos da MDA 4

4 Ferramenta Modelio **6**

5 Desenvolvimento do Módulo Kuaba **8**

5.1 Idealização, ícones e imagens 9

5.2 Perfil Kuaba - Perfil para o Modelo de Discussão 9

5.3 Definição de Diagramas e Elementos MDE 12

5.4 Aba de Propriedades e *wizard* de criação 15

5.5 Transformação de CIM para PIM 21

6 Exemplo de uso **24**

7 Considerações finais **31**

REFERÊNCIAS **32**

1 Introdução

Este estudo de iniciação científica integra o projeto de pesquisa intitulado "Suporte a Design Rationale em Engenharia de Software dirigida por Modelos usando a abordagem Kuaba". O objetivo principal é facilitar a implementação de modelos de discussão (*design rationale*) gerados por meio da metodologia Kuaba em ferramentas de Engenharia Dirigida por Modelos (MDE - *Model-Driven Engineering*), apresentando-se como uma solução promissora para um dos desafios técnicos em MDE destacados por (Bucchiarone *et al.*, 2020).

O foco deste trabalho está na concepção e desenvolvimento de uma interface gráfica de usuário dentro do contexto da MDE, destinada a auxiliar na captura do *design rationale*. Esse projeto adota a abordagem Kuaba, complementada pelo perfil DDD definido por (Rademacher; Sachweh; Zündorf, 2018), como fundamento para suas atividades de pesquisa.

A implementação bem-sucedida desta solução dependeu de uma série de fatores, incluindo o entendimento sobre a abordagem Kuaba, o estudo de mecanismos de extensão para a ferramenta MDE escolhida, a idealização e criação de elementos visuais eficazes, a implementação das funcionalidades de acordo com a proposta do projeto e, por fim, a realização de testes e a disponibilização da extensão para uso prático.

2 Design Rationale

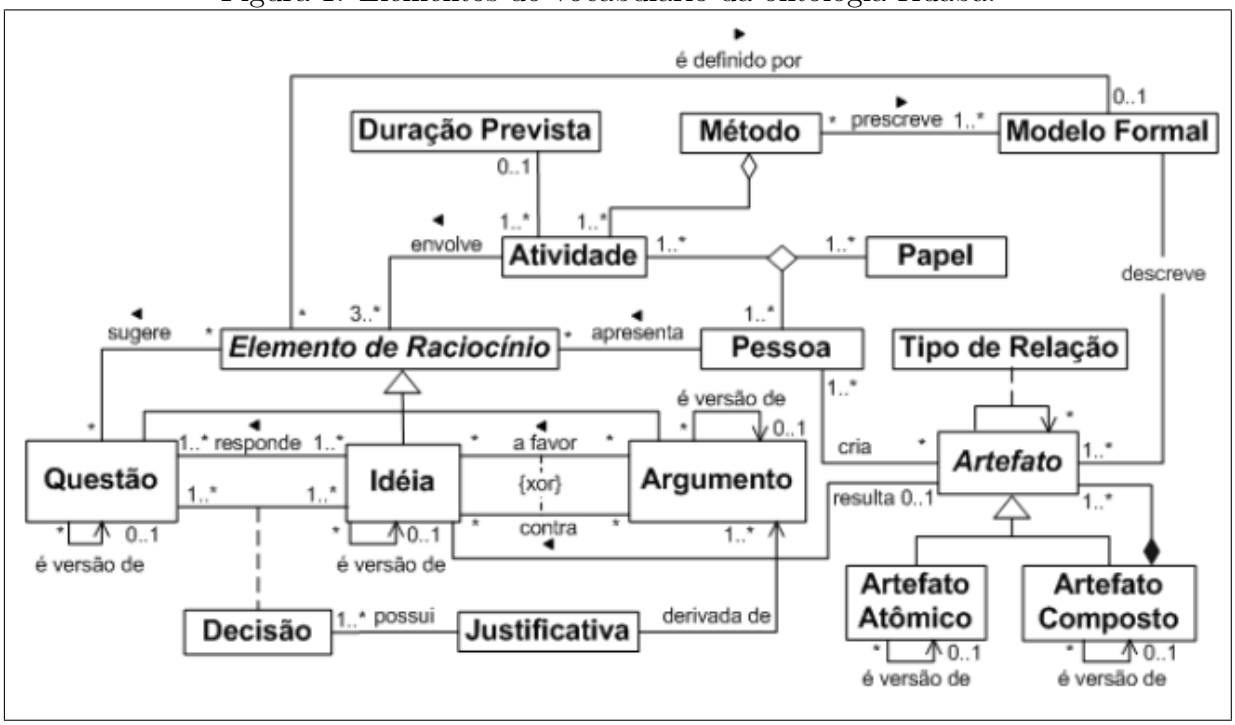
Design rationale é uma abordagem de representação do conhecimento, que serve para registrar diversos elementos relacionados a escolhas de design. Registra elementos como os motivos e as justificativas para uma decisão de design, argumentações para tal decisão e todas as possibilidades consideradas (Lee, 1997).

2.1 Ontologia Kuaba

A ontologia Kuaba é um modelo de representação de conhecimento para o domínio de *design rationale*. Seu objetivo é estabelecer um vocabulário que atribua significado ao seu conteúdo registrado e definir um conjunto de regras que permitam inferências e operações computáveis para apoiar o uso do mesmo. A ontologia foi proposta com este intuito, representar o *design rationale* de forma precisa e computável, e assim garantir que o conhecimento registrado durante o design possa ser processado automaticamente (Medeiros, 2006).

A ontologia apresenta um vocabulário bem definido, composto por uma variedade de elementos destinados a representar o *design rationale*. Este vocabulário inclui os elementos de raciocínio, Questão, que representa o problema a ser resolvido, Ideia, que representa possiveis soluções para o problema, e Argumento, que representa argumentos contra ou a favor a possível solução. Além disso, define outros elementos, como Decisão que representa as decisões tomadas para os elementos de raciocío Ideia, entre outras representações como artefatos e atividades de design (Medeiros, 2006), como ilustra a Figura 1.

Figura 1: Elementos do vocabulário da ontologia Kuaba.



Fonte: (Medeiros, 2006)

A Abordagem Kuaba viabiliza a captura, representação e processamento do *design rationale*, com o intuito de fomentar o reúso de designs com base em modelos. Ela se vale da semântica incorporada ao metamodelo adotado no processo de design para instanciar os elementos fundamentais da ontologia. Ao unir a abordagem Kuaba com soluções MDE, pode-se melhorar a compreensão dos modelos e a possibilidade de simular diferentes soluções de design. Além de facilitar a colaboração entre desenvolvedores ao tornar explícitos os elementos de decisão e seus argumentos (Medeiros, 2020).

3 *Model-Driven Architecture* - MDA

A MDA foi desenvolvida pela *Object Management Group* (OMG) visando a resolução de problemas de integração. Ela estabelece uma abordagem para a especificação de sistemas de TI que separa as especificações da funcionalidade do sistema e de como essa funcionalidade é implementada em uma plataforma tecnológica específica (como uma linguagem de programação, por exemplo). Com essa finalidade, a MDA estabelece uma arquitetura para modelos que oferece um conjunto de diretrizes para estruturar especificações expressas como modelos (OMG, 2001).

A abordagem MDA, juntamente com os padrões que a respaldam, possibilita que o mesmo modelo que define a funcionalidade do sistema seja implementado em múltiplas plataformas, seja através de padrões de mapeamento auxiliares ou por meio de mapeamentos diretos para plataformas específicas. Isso também viabiliza a integração de diferentes aplicativos, ao relacionar de forma explicita seus modelos, promovendo a integração e interoperabilidade. Além disso, oferece suporte a evolução do sistema, adaptando-se às mudanças tecnológicas das plataformas ao longo do tempo (OMG, 2001).

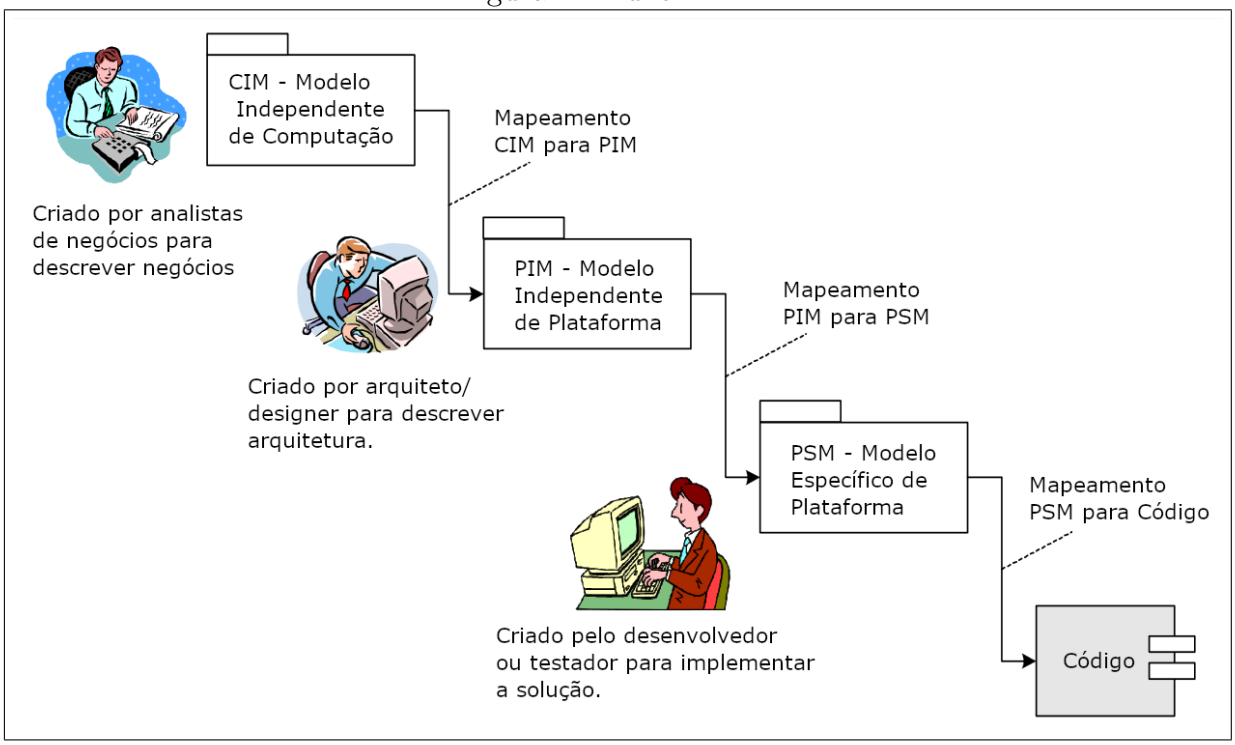
3.1 Modelos da MDA

A MDA determina uma estrutura consistente de modelos, os quais representam parte da função, estrutura e/ou comportamento de um sistema, e possui três categorias de modelos. Estas categorias são os Modelos Independentes de Computação (CIM - *Computational Independant Model*) Modelos Independentes de Plataforma (PIM - *Platform Independent Models*) e os Modelos Específicos de Plataforma (PSM - *Platform Specific Model*). Uma Plataforma é definida como uma infraestrutura de *software* implementada com uma tecnologia específica em uma tecnologia de *hardware* específica (OMG, 2001).

O PIM fornece especificações formais da estrutura e função do sistema, abstraindo detalhes técnicos, enquanto o PSM é expresso em termos do modelo de especificação de uma plataforma alvo, utilizando conceitos específicos para a plataforma escolhida. Entre

esses modelos, são realizadas transformações através de um recurso importante para a abordagem, os mapeamentos. Esses mapeamentos são conjuntos de regras e técnicas utilizadas para modificar um modelo com intuito de obter outro modelo (OMG, 2001). Eles são empregados em transformações de modelo para modelo, como a transformação de PIM para PSM, e de modelo para artefato (o inverso também é possível), como um PSM para código Java. Já o CIM é um modelo de negócios, que apenas descreve conceitos de negócios com foco em coisas reais, não das representações delas em sistemas de informação (OMG, 2014). A Figura 2 mostra o fluxo da MDA.

Figura 2: Fluxo MDA



Fonte: traduzida de (David S. Frankel *et al.*, 2003)

4 Ferramenta Modelio

A Modelio é um ambiente de modelagem de código aberto, que oferece uma ampla variedade de funcionalidades com base nos principais padrões de arquitetura empresarial, desenvolvimento de software e engenharia de sistemas e possui suporte a uma diversidade de modelos e diagramas, fornecendo muitos serviços que facilitam a modelagem de arquiteturas, como recursos de verificação de consistência do modelo. É uma ferramenta dedicada a desenvolvedores, analistas, arquitetos de software e sistemas (Modelio, 2024).

Alguns recursos do Modelio o tornaram a escolha ideal para este projeto. Ele apresenta um sistema de extensão onde as funcionalidades são empacotadas na forma de módulos, os quais são mecanismos flexíveis que agrupam todos os artefatos de uma extensão do Modelio, como código, perfis, ícones e documentação. Os módulos podem ser facilmente adicionados, removidos ou combinados com outros a qualquer momento em um projeto do Modelio, permitindo que o usuário adapte suas ferramentas selecionando ou desmarcando-os (Modeliosoft, 2024a). Esse sistema serviu como base para este projeto.

Outro aspecto crucial da ferramenta é o módulo chamado *JavaDesigner*, que oferece suporte à geração e reversão de código Java a partir de modelos. Isso é essencial para o projeto, especialmente na geração de código Java a partir de um modelo (Modelio, 2024).

Além disso, a Modelio possui outros recursos relevantes para a MDA. Seu forte suporte a perfis permite a aplicação, remoção ou combinação dinâmica de perfis. A ferramenta utiliza o metamodelo UML, que foi adaptado com intuito de reforçar sua consistência e simplificar o uso, além de aproveitar ao máximo a orientação a objetos durante a manipulação por parte dos programadores. Além disso, todas as operações manuais realizadas no Modelio podem ser realizadas programaticamente. Isso inclui transformação de modelos, criação de diagramas e aplicação de padrões, que podem ser facilmente programados utilizando a API. Novos geradores também podem ser desenvolvidos utilizando serviços predefinidos para engenharia de ida e volta. O editor da Modelio pode ser personalizado adicionando novos tipos de diagramas, uma nova interface gráfica do usuário ou novos

acoplamentos a outras ferramentas (Modeliosoft, 2024b).

5 Desenvolvimento do Módulo Kuaba

Tendo em vista os recursos da Modelio, foi utilizado seu mecanismo de extensão para a implementação da solução MDE utilizando a abordagem Kuaba, que ocorreu por meio do desenvolvimento de um módulo para a ferramenta. O módulo foi pensado para ser um ambiente separado para se trabalhar com Engenharia Dirigida a Modelos. O módulo contém o CIM, que será representado por um modelo de discusão seguindo a ontologia Kuaba, o PIM e o PSM, além de contemplar as transformações automáticas entre essas partes, incluindo a geração de código.

Foi necessário configurar a ferramenta utilizada, neste caso o Eclipse IDE, para dar início ao desenvolvimento. Os passos descritos no *Module Developer Guide*[1], disponível na documentação da Modelio no GitHub, foram seguidos para importar o *Template* do Módulo. O *Template* fornecido pela Modelio serviu como ponto de partida, apresentando a estrutura base do projeto e as bibliotecas necessárias.

A concepção deste projeto foi organizada seguindo uma ordem bem definida, porém com certo dinamismo. O primeiro passo foi a idealização e criação dos ícones e imagens para os elementos gráficos, depois a criação de um perfil para dar suporte à criação de modelos de discussão e permitir a representação visual dos elementos dos modelos de acordo com a ontologia Kuaba. Após isso, ocorreu a implementação dos recursos visuais na ferramenta e algumas funcionalidades, como a definição dos diagramas e elementos MDE personalizados. Também foram realizadas alterações no menu contextual da ferramenta e a implementação de uma nova aba de propriedades. Por fim, foi elaborada a transformação do CIM para o PIM. Já as transformações de PIM para PSM e a geração de código, são escopo de outro trabalho no mesmo projeto (Cunha, 2023), em relação a estas, apenas a estrutura foi implementada para receber tais transformações.

[1] https://github.com/ModelioOpenSource/Modelio/wiki/Module-Developer-Guide-Index

5.1 Idealização, ícones e imagens

Em primeiro momento, foram idealizados os ícones com propósito de serem visualmente atraentes, mas principalmente transmitir a ideia do que cada elemento representa, de forma rápida. Os elementos de raciocínio foram definidos da seguinte forma: o elemento Ideia foi representado com a imagem de uma lâmpada acesa, a Questão pela imagem de um ponto de interrogação e o Argumento pela imagem de duas pessoas lado a lado com caixas de diálogo. Já as relações entre os elementos foram representadas assim: a relação *isAddressedBy* por uma seta contínua, as relações *ObjectsTo* e *inFavorOf* com uma seta tracejada com a letra "O" no canto superior esquerdo e uma seta tracejada com a letra "F" no canto superior esquerdo, respectivamente. Por fim, a relação *Suggests* foi representada por uma seta tracejada e pontilhada de cor cinza, com a letra "S" no canto superior esquerdo.

Após isso, as imagens e ícones foram criadas conforme a demanda do desenvolvimento. Desde imagens para preview na criação de diagramas, assim como diversos ícones relacionados as partes do projeto, mostrados na Figura 3.

Figura 3: Ícones criados

5.2 Perfil Kuaba - Perfil para o Modelo de Discussão

Após a criação dos ícones príncipais, foi necessário definir um perfil UML para dar suporte à criação de modelos de discussão e permitir a representação gráfica do diagrama e seus elementos, seguindo a ontologia Kuaba.

O desenvolvimento do perfil foi efetuado com as devidas modificações no arquivo "module.xml". A criação do mesmo envolve a definição de estereótipos, especificação de metaclasses que são estendidas pelos estereótipos e a inserção de ícones e propriedades para cada elemento, conforme mostrado no exemplo na Figura 4.

Kuaba-Software-Engineering-tool - module.xml

```xml
<Profiles>
    <Profile uid="" id="KuabaProfile">
        <Stereotype name="Question" metaclass="Class" owner-stereotype="reasoningElement" is-hidden="false" label="Kuaba Question" uid="">
            <Icon path="res/icons/question_24.png"/>
            <Image path="res/icons/question_48.png"/>
            <TagTypes>
                <TagType name="Type" uid="" label="Type" parameter-card="1" is-hidden="true" is-signed="false"/>
            </TagTypes>
        </Stereotype>
    </Profile>
</Profiles>
```

Figura 4: Exemplo de Definição de Perfil e Estereótipos

A configuração do perfil é realizada por meio da *tag Profile*. É necessário definir um ID para o perfil, enquanto o UID é automaticamente definido durante a compilação do módulo. Dentro desta *tag*, deve-se definir os estereótipos, através da *tag Stereotype*, passando como parâmetros o nome (que será usado como identificador neste arquivo de configuração), a metaclasse que o estereótipo irá estender, qual estereótipo irá estender (se for o caso), se será visível para o usuário pela ferramenta e o rótulo que será exibido na ferramenta. Dentro da *tag Stereotype* serão passadas outras *tags* para determinar o ícone e a imagem, além das *TagTypes* responsáveis por criar propriedades para o estereótipo. Elas possuem parâmetros como identificador, nome de exibição, o tipo que a propriedade irá receber, como binário, texto e seletor, entre outros.

O perfil Kuaba foi elaborado conforme mostra a Figura 5, estendendo as metaclasses *Class*, *Association*, *Dependency* e *StaticDiagram* da UML e definindo os estereótipos necessários para o diagrama, os elementos de raciocínio, as relações entre eles e suas propriedades.

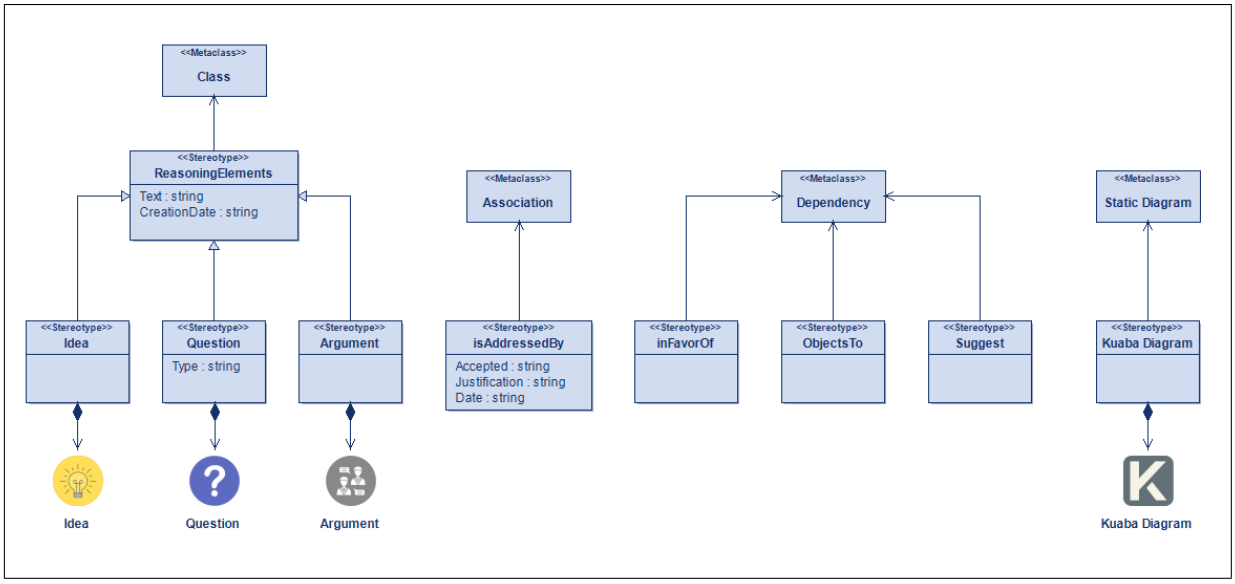


Figura 5: Perfil Kuaba

Os estereótipos dos elementos de raciocínio estendem a metaclasse *Class* e incluem propiedades *Text*, para armazenar o texto referente ao elemento, e *CreationDate*, que registra a data e hora de criação do elemento. Dois desses elementos, *Idea* e *Argument*, possuem apenas as propriedades herdadas, enquanto o terceiro, *Question*, possui também a propriedade *Type* com os possíveis valores XOR, AND e OR, para indicar se a aceitação de ideias para a questão devem ser mutuamente exclusivas ou não.

As relações entre os elementos de raciocínio estendem as metaclasses *Association* e *Dependency*. A primeira é estendida apenas pela relação *isAddressedBy*, que relaciona os elementos *Question* e *Idea*, onde uma *Question* é respondida por uma *Idea*. Nesta relação são definidas as propriedades *Decision*, que determina se a ideia foi aceita ou não, podendo receber os valores *accepted* ou *rejected*, *Justification*, que recebe um texto como justificativa para a decisão tomada para a propriedade anterior, e *Date*, que registra a data e hora que a ideia foi aceita ou rejeitada.

As relações *inFavorOf*, *ObjectsTo* e *Suggests* estendem a metaclasse *Dependency* e não possuem propriedades. *inFavorOf* e *ObjectsTo* relacionam os elementos *Argument* e *Idea*, indicando que um argumento pode ser a favor ou contra uma ou mais ideias. Já a relação *Suggests*, é uma relação que liga qualquer elemento de raciocínio a uma questão, sendo assim, qualquer elemento pode sugerir uma questão.

5.3 Definição de Diagramas e Elementos MDE

Com o perfil Kuaba definido, o próximo passo foi o desenvolvimento dos diagramas e dos elementos necessários para os processos da MDE. Para isso foi preciso criar novos estereótipos para estes diagramas e elementos, conforme demonstrado no tópico anterior.

Primeiramente, foi implementada a parte estrutural. O Módulo foi pensado para funcionar de forma independente dos padrões da Modelio, então foi criada uma estrutura de pacotes para os modelos previstos e seus diagramas, além de determinar escopos bem definidos para garantir que os elementos e suas funcionalidades funcionem apenas onde devem. O pacote Kuaba é o pacote raiz do módulo, onde todos os elementos do Módulo Kuaba podem e devem ser criados. Já os pacotes CIM, PIM e PSM serão utilizados dentro pacote raiz e determinam os escopos dos diagramas, elementos e transformações relacionadas a eles.

Para incluir a estrutura de pacotes, foram adicionadas novas configurações no arquivo "module.xml", além dos estereótipos. A criação dos pacotes pelo usuário será feita pelo menu contextual e, para inserir novos comandos no menu, é necessário acresentar a *tag Commands* para a lista de comandos e *Command* para especificar um comando, como mostrado na Figura 6. Na *tag Scope*, é definido o escopo do comando, passando a metaclasse e o estereótipo do elemento em que o comando deverá ser utilizado, neste caso um pacote com estereótipo de *KuabaPackage*. Para definir a ação do comando, a *tag Handler* é utilizada, passando uma classe que será executada pelo comando. Essa classe pode ser uma classe implementada ou uma genérica, como *ElementCreate*, que cria elementos com os parâmetros passados pelas *tags HParameter*, como nome do elemento, metaclasse e estereótipo. Após isso, é definido o menu contextual a partir da *tag Contextual Menu*, passando as referências dos comandos criados e agrupamentos de comandos.

Kuaba-Software-Engineering-tool - module.xml

```xml
<Gui>
    <Commands>
        <Command id="CreateCIMPackage" label="Create a CIM Package" tooltip="Create a Discussion Model (CIM) Package" image="res/icons/cim_package_24.png">
            <Scope metaclass="Package" stereotype="KuabaPackage"/>
            <Handler class="ElementCreate">
                <HParameter name="name" value="CIM Package"/>
                <HParameter name="metaclass" value="Package"/>
                <HParameter name="stereotype" value="CIMPackage"/>
            </Handler>
        </Command>
    </Commands>
    <ContextualMenu>
        <CommandRef refid="CreateCIMPackage"/>
        <CommandRef refid="referência" group="Rótulo para grupo" group-image="imagem para grupo"/>
    </ContextualMenu>
</Gui>
```

Figura 6: Configuração de novos Comandos

Após a configuração da estrutura de pacotes, foram configurados comandos para a criação dos diagramas Kuaba e PIM, seguindo o formato da Figura 6, porém utilizando a classe pré-definida *DiagramCreate* e passando as informações dos diagramas como parâmetros. Além disso, foi necessário configurar as individualidades dos diagramas, como exibe a Figura 7. A *tag DiagramType* dentro de *Diagrams* é utilizada para definir o diagrama base, como *StaticDiagram* para o diagrama Kuaba e *ClassDiagram* para o PIM, além do estereótipo, neste caso o definido pelo perfil Kuaba. A paleta de elementos que o diagrama poderá usar é definida pela *tag Palette*, onde é possivel utilizar a paleta do diagrama base ou criar uma nova paleta com novos elementos. A *tag Handler* lida com a customização do diagrama, recebendo uma classe e parâmetros.

Kuaba-Software-Engineering-tool - module.xml

```
1  <Gui>
2      <Diagrams>
3          <DiagramType base-diagram="StaticDiagram" stereotype="KuabaDiagram">
4              <Palette keepBasePalette="false">
5                  <ToolRef refid="CreateQuestionByPalette" group="Reasoning Elements"/>
6                  <ToolRef refid="CreateIdeaByPalette" group="Reasoning Elements"/>
7                  <ToolRef refid="CreateArgumentByPalette" group="Reasoning Elements"/>
8                  <ToolRef refid="CreateIsAddressedByLink" group="Decision link"/>
9                  <ToolRef refid="CreateInFavorOfLink" group="Argument links"/>
10                 <ToolRef refid="CreateObjectsToLink" group="Argument links"/>
11                 <ToolRef refid="CreateSuggestionLink" group="Suggestion link"/>
12             </Palette>
13             <Handler class="StandardCustomizer">
14                 <HParameter name="stereotype" value="KuabaDiagram"/>
15                 <HParameter name="metaclass" value="StaticDiagram"/>
16             </Handler>
17         </DiagramType>
18     </Diagrams>
19 </Gui>
```

Figura 7: Configuração de novos Diagramas

Para o Diagrama Kuaba em especifico, foi indispensável a criação de novos elementos de paleta que seguissem a abordagem Kuaba. Elementos de paleta são definidos através da *tag Tools*, onde são indicados os itens identificador, rótulo, a dica que aparecerá quando o usuário passar o *mouse* no elemento e a imagem do mesmo. Além desses items, também são definidos o escopo e a classe com seus parâmetros (Figura 8). Ao fim do processo, os novos elementos de paleta criados são recuperados como referência pela *tag ToolRef* dentro da *tag Palette*, como na Figura 7.

Kuaba-Software-Engineering-tool - module.xml

```xml
<Gui>
    <Tools>
        <Tool id="CreateQuestionByPalette" label="Create a Question element" tooltip="Create a new Question element" image="res/icons/question_24.png">
            <Scope-source metaclass="ModelTree"/>
            <Handler class="Box">
                <HParameter name="name" value="Question"/>
                <HParameter name="metaclass" value="Class"/>
                <HParameter name="stereotype" value="Question"/>
            </Handler>
        </Tool>
    </Tools>
</Gui>
```

Figura 8: Configuração de novos elementos de paleta

Com a possibilidade de criação dos diagramas a partir do menu contextual, o foco posterior foi a estilização dos elementos dos diagramas. Para isso, alterações foram detalhadas nos arquivos ".style", seguindo um padrão já estabelecido para o projeto. Esses arquivos contêm as nomenclaturas correspondentes a cada metaclasse UML, indicando a propriedade de estilo a ser modificada e suas opções de estilização correspondentes. Então se um elemento com a metaclasse *dependency* estiver com alguma propriedade de estilo modificada, a alteração será realizada para todos os elementos com a mesma metaclasse, independentemente de terem estereótipos diferentes.

Poucos dos elementos Kuaba precisaram de classes próprias para criação e estilização, devido a limitação do arquivo ".style". Os elementos em questão foram *ObjectsTo*, *inFavorOf* e *Suggests*, por serem diferentes estereótipos que utilizam a mesma metaclasse da UML, a *Dependency*. As classes citadas foram implementadas de forma semelhante, estendem a classe *DefaultLinkTool* da API da Modelio e sobrescreve três métodos. Os dois primeiros, *acceptFirstElement* e *acceptSecondElement* são responáveis por verificar se o elemento inicial e o elemento final, do *link*, selecionados pelo usuário são permitidos. Com a permissão para criação o método *actionPerformed* é chamado, o mesmo recupera a sessão atual, o elemento de origem e o elemento alvo, depois cria o *link* especificando a rota e aplica as estilizações individuais (Figura 9). Após serem implementadas foram adicionadas na classe da *tag Handler* dentro das *tags Tool* dos respectivos elementos.

Kuaba-Software-Engineering-tool - InFavorOfLinkCommand.java

```java
@Override
public void actionPerformed(IDiagramHandle diagramHandle, IDiagramGraphic originNode, IDiagramGraphic targetNode, LinkRouterKind routerType, ILinkRoute path) {

    ModelElement originElement = (ModelElement) originNode.getElement();
    ModelElement targetElement = (ModelElement) targetNode.getElement();

    IModelingSession session = KuabaModule.getInstance().getModuleContext().getModelingSession();

    try (ITransaction transaction = session.createTransaction("Link Creation")) {

        Dependency newLink = session.getModel().createDependency(originElement, targetElement, "KuabaModule", "inFavorOf");
        newLink.setName("InFavorOf");

        List<IDiagramGraphic> diagramHandler = diagramHandle.unmask(newLink, 0, 0);
        IDiagramLink link = (IDiagramLink) diagramHandler.get(0);
        link.setRouterKind(routerType);
        link.setRoute(path);
        diagramHandle.save();

        link.setLinePattern(1);
        link.setLineRadius(5);
        link.setLineColor("32,178,170");
        transaction.commit();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Figura 9: Classe de criação e estilização de Links

5.4 Aba de Propriedades e *wizard* de criação

Posteriormente foram idealizadas a aba de propriedades e a criação do diagrama a partir do *Wizard* de criação, ambas com a finalidade de simplificar e aprimorar a experiência do usuário, segregando os conteúdos relacionados ao Módulo.

O *Wizard* de criação é uma tela acionada a partir da opção *Create diagram* no menu contextual já disponível na ferramenta Modelio. Esta tela exibe opções para criação de todos os diagramas, independente do módulo em que estejam, porém respeitando o escopo de cada módulo. Para inclusão dos Diagramas do Módulo Kuaba no *Wizard* de criação, foi necessária a criação de uma *tag Wizard* dentro das *tags DiagramType* correspondentes aos diagramas, como na Figura 10.

Kuaba-Software-Engineering-tool - module.xml

```xml
<DiagramType base-diagram="ClassDiagram" stereotype="PIMDiagram">
    <Palette keepBasePalette="true"/>
    <Wizard label="PIM Diagram" icon="res/icons/kuaba_pim2_24.png" information="test info" details="test details" preview-image="res/images/pim_preview.png">
        <Scope metaclass="Package" stereotype="PIMPackage"/>
        <Handler class="kuaba.Module.wizard.PIMDiagramWizardContributor">
            <HParameter name="style" value="PIMPSMStyle"/>
        </Handler>
    </Wizard>
    <Handler class="StandardCustomizer">
        <HParameter name="stereotype" value="PIMDiagram"/>
        <HParameter name="metaclass" value="ClassDiagram"/>
    </Handler>
</DiagramType>
```

Figura 10: Configuração do Wizard para o diagrama

Assim é possível definir o rótulo, o ícone, uma breve informação sobre a ação realizada,

uma descrição para o diagrama, uma imagem de pré-visualização e o escopo do diagrama. O resultado da inserção fica como a Figura 11.

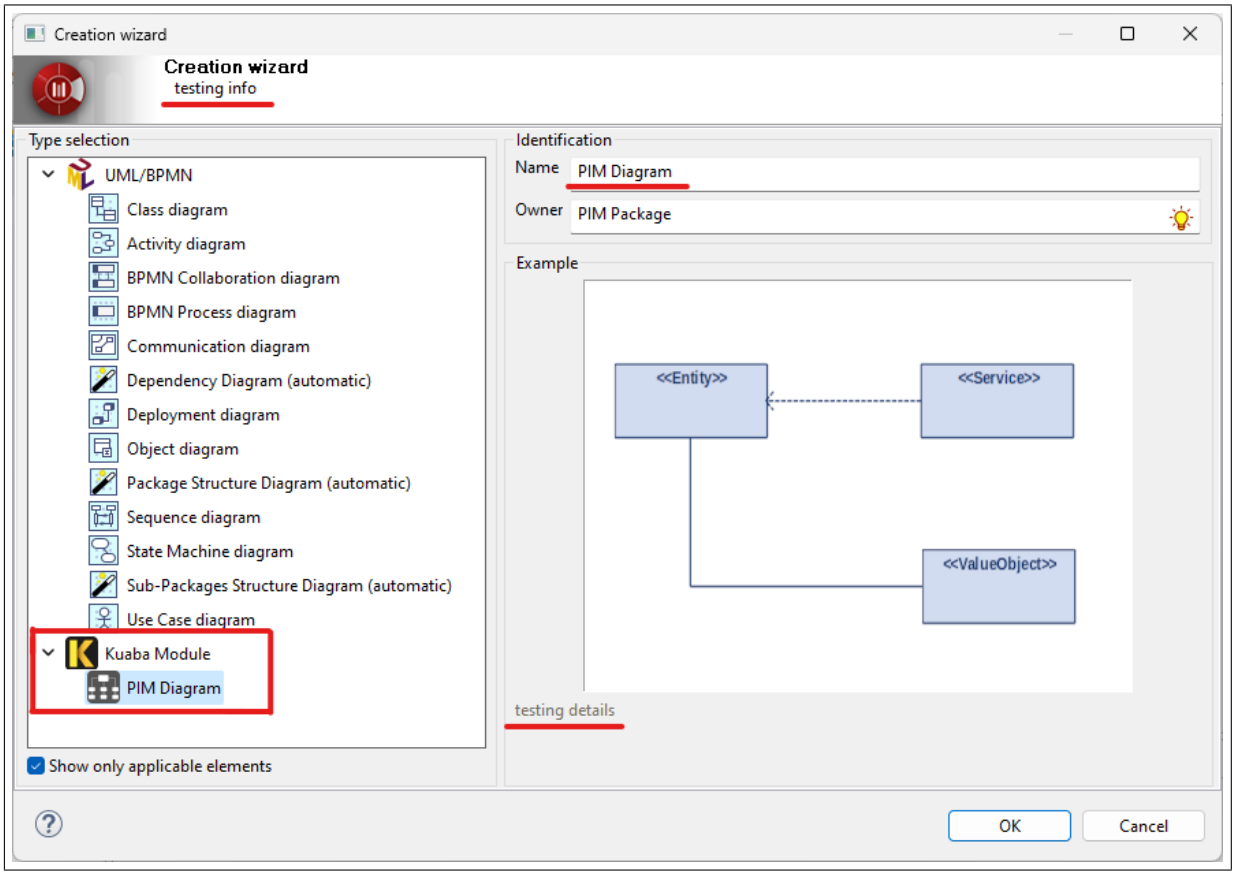


Figura 11: Wizard de Criação

Porém para efetivar tal inclusão, foi necessária a implementação de novas classes, essas classes foram divididas em dois grupos, as classes para apoiar a criação dos diagramas e as classes destinadas a manipular o *Wizard*. As relações entre estas classes são ilustradas na Figura 12

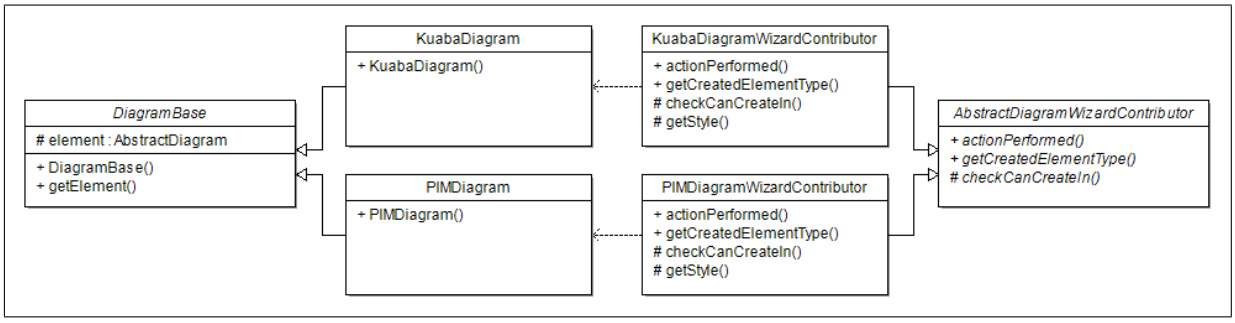


Figura 12: Relacionamento das classes para o Wizard de criação

Dessa maneira, foram desenvolvidas as classes *DiagramBase*, *KuabaDiagram* e *PIM-*

Diagram. A classe abstrata DiagramBase é uma base para a criação dos diagramas, ela possui um atributo protegido do tipo AbstractDiagram que guarda uma referência para o diagrama que será criado. Nela há o método *getElement* que recupera o valor do atributo citado e o método construtor, que é utilizado pelas classes *DiagramBase*, *KuabaDiagram* que a estendem. Como o exibido na Figura 13, este construtor recebe as informações dos diagramas, como elemento pai, estilos, estereótipo, nome e metaclasse. Ele é responsável por recuperar a sessão de usuário e o estereótipo do diagrama, indentificando qual deve ser criado e efetuando a criação, após isso, aplica a estilização indicada e salva.

Kuaba-Software-Engineering-tool - DiagramBase.java

```
1  public DiagramBase(ModelElement owner, IStyleHandle style, IModuleContext context, String stereotypeName, String diagramName, Class<? extends MObject> diagramClass) throws Exception {
2      try {
3          IModelingSession session = context.getModelingSession();
4          Stereotype stereotype = session.getMetamodelExtensions()
5              .getStereotype(IKuabaPeerModule.MODULE_NAME, stereotypeName, context.getModelioServices().getMetamodelService().getMetamodel().getMClass(diagramClass));
6
7          if(diagramClass.equals(StaticDiagram.class)) {
8              this.element = session.getModel().createStaticDiagram(diagramName, owner, stereotype);
9          }
10         else if(diagramClass.equals(ClassDiagram.class)) {
11             this.element = session.getModel().createClassDiagram(diagramName, owner, stereotype);
12         }
13
14         try (IDiagramHandle diagramHandler = context.getModelioServices().getDiagramService().getDiagramHandle(getElement())) {
15             diagramHandler.getDiagramNode().setStyle(style);
16             diagramHandler.save();
17             diagramHandler.close();
18         }
19
20     } catch (Exception e) {
21         e.printStackTrace();
22     }
23
24     IModelChangeHandler modelHandler = (IModelChangeHandler) new ModelChangeHandler();
25     context.getModelingSession().addModelHandler(modelHandler);
26 }
```

Figura 13: Método construtor da classe DiagramBase

O segundo grupo de classes, contém as classes *KuabaDiagramWizardContributor* e *PIMDiagramWizardContributor* que estendem a classe abstrata *AbstractDiagramWizardContributor* da API da Modelio. As classes desenvolvidas são bem semelhantes, elas sobreescrevem os métodos herdados *actionPerformed*, *getCreatedElementType* e *checkCanCreateIn*. O método *checkCanCreateIn* apenas verifica se o elemento tem permissão para ser criado no local selecionado. Já o *getCreatedElementType* retorna um *ElementDescriptor*, representando o tipo do elemento, recebendo sua metaclasse e o estereótipo. Por fim, o método *actionPerformed*, que de fato cria o diagrama instanciando as respectivas classes dos diagramas e retornando o mesmo.

Kuaba-Software-Engineering-tool - PIMDiagramWizardContributor.java

```
1  @Override
2  public AbstractDiagram actionPerformed(ModelElement diagramOwner, String diagramName, String diagramDescription) {
3      IModuleContext context = getModule().getModuleContext();
4      IModelingSession session = context.getModelingSession();
5      IDiagramService diagramService = context.getModelioServices().getDiagramService();
6
7      try (ITransaction transaction = session.createTransaction("");) {
8          IStyleHandle style = diagramService.getStyle(getStyle());
9          PIMDiagram pim = new PIMDiagram(diagramOwner, style, context);
10         pim.getElement().setName(diagramName);
11         context.getModelioServices().getEditionService().openEditor(pim.getElement());
12         transaction.commit();
13         return pim.getElement();
14
15     } catch (Exception e) {
16         context.getLogService().error(e);
17     }
18     return null;
19 }
```

Figura 14: Método *actionPerformed* da classe *PIMDiagramWizardContributor*

A adição no *Wizard* de criação assegura a integração dos Diagramas Kuaba no conjunto de opções de criação de diagramas, proporcionando ao usuário a capacidade de definir, visualizar e selecionar esse tipo específico de diagrama de maneira mais coerente com a ferramenta Modelio.

O próximo recurso desenvolvido foi a Aba de Propriedades, uma adição que possibilitou a modificação direta das propriedades dos estereótipos do elemento selecionado. Essa funcionalidade remove elementos visuais e opções desnecessárias, ao mesmo tempo em que separa o escopo do módulo. Essa abordagem facilita o uso do diagrama Kuaba, permitindo a alteração de aspectos importantes de cada elemento para realizar transformações. A aba personalizada pode ser vista na Figura 15.

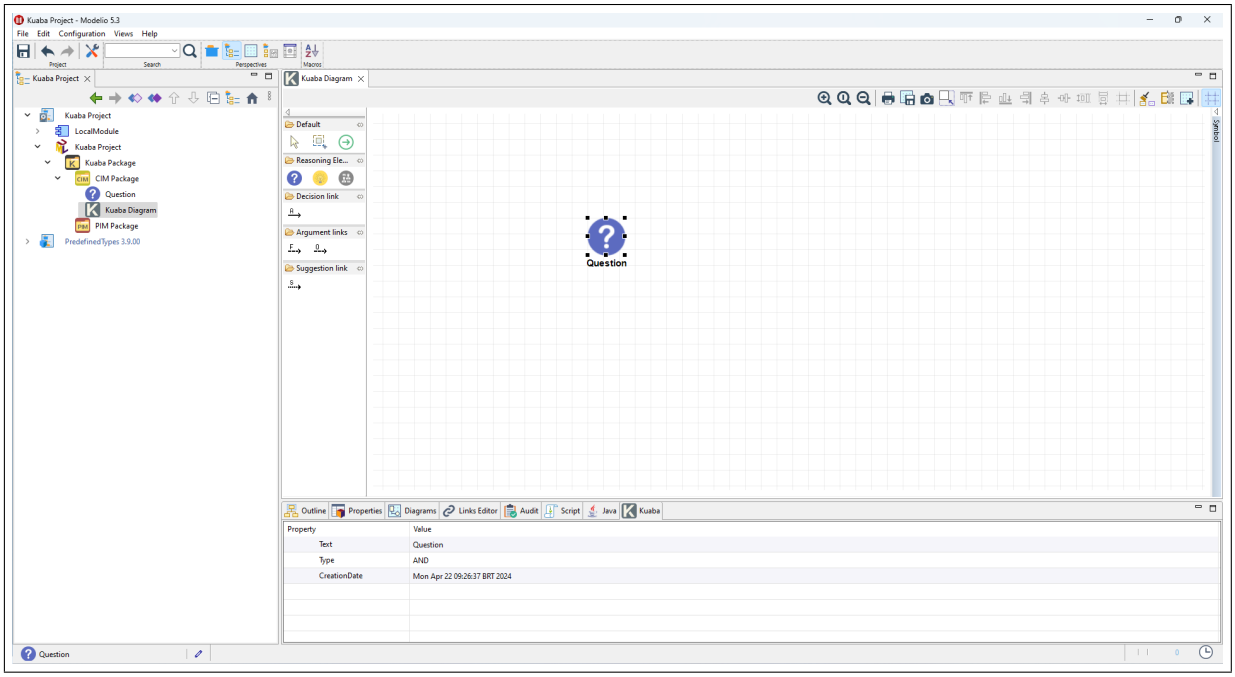


Figura 15: Aba de Propriedades Personalizada

Para a implementação da aba de propriedades foi preciso, no arquivo de configuração, utilizar apenas a *tag PropertyPage* dentro da *tag Views* para especificar propriedades como o identificador, o rótulo, o ícone da aba e a classe responsável por manipulá-la. Em seguida foram desenvolvidas diversas classes, ilustradas na Figura 16.

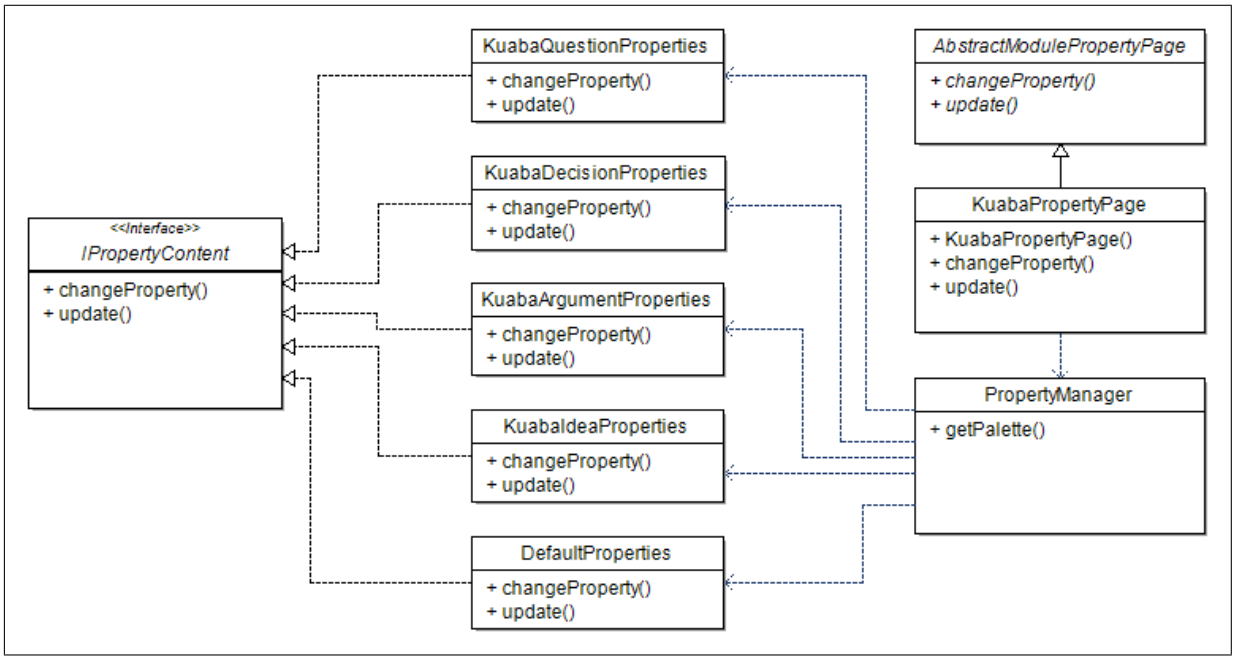


Figura 16: Hierarquia da Aba de Propriedades

A classe principal que lida diretamente com as alterações na aba é a *KuabaPropertyPage*, que estende a classe *AbstractModulePropertyPage* e sobreescreve os métodos herdados *changeProperty* e *update*, e possui um construtor que chama o construtor da classe pai. O método *changeProperty* é responsável por verificar se houve uma alteração no valor de alguma propriedade na aba. O método *update* é utilizado para atualizar o que está sendo exibido na aba. Quando o usuário seleciona um elemento diferente, este método recupera as propriedades relativas ao novo elemento e exibe na aba.

Adicionalmente, foram definidas uma interface e classes para o gerenciamento e detalhamento dos elementos da aba. A interface *IPropertyContent* possui as assinaturas dos mesmos métodos citados anteriormente, *changeProperty* e *update*. Porém com uma modificação em um parâmetro, que antes recebia uma lista e agora recebe um único elemento. Esta interface é implementada pelas classes que definem as propriedades da aba para os elementos da ontologia Kuaba. Nas classes, o método *changeProperty* verifica qual propriedade do elemento foi modificada na aba e altera o valor correspondente no estereótipo. Por outro lado, o método *update* sincroniza as propriedades da aba com os valores definidos no estereótipo no momento em que são exibidas na mesma. Por fim, a classe de gerenciamento *PropertyManager* simplesmente localiza o elemento selecionado, identifica seu estereótipo e retorna uma nova instância da classe de aba associada a esse elemento. A Figura 17, mostra um código de exemplo para definição das propriedades na aba.

Kuaba-Software-Engineering-tool - KuabaQuestionProperties.java

```
1  public class KuabaQuestionProperties implements IPropertyContent {
2
3      @Override
4      public void changeProperty(ModelElement element, int row, String value) {
5          if (row == 1) {
6              element.setName(value);
7              Utils.setPropertyValue(KuabaPeerModule.MODULE_NAME, "Text", value, element);
8          }
9          if (row == 2)
10             Utils.setPropertyValue(KuabaPeerModule.MODULE_NAME, "Type", value, element);
11         if (row == 3)
12             Utils.setPropertyValue(KuabaPeerModule.MODULE_NAME, "CrationDate", value, element);
13     }
14
15     @Override
16     public void update(ModelElement element, IModulePropertyTable table) {
17         table.addProperty("Text", element.getName());
18         String[] typePropertyValues = {"XOR", "AND", "OR"};
19         table.addProperty("Type", Utils.getPropertyValue("Type", element), typePropertyValues);
20         if (Utils.getPropertyValue("CreationDate", element) == "") {
21             table.addProperty("CreationDate", Utils.setDate(KuabaPeerModule.MODULE_NAME ,"CreationDate", element));
22         } else {
23             table.addProperty("CreationDate", Utils.getPropertyValue("CreationDate", element));
24         }
25     }
26 }
```

Figura 17: Código das propriedades do elemento Questão

5.5 Transformação de CIM para PIM

Neste trabalho foi implementada a transformação de CIM para PIM, neste caso de um diagrama Kuaba (uma forma de representar o CIM) para um Diagrama PIM. A transformação foi realizada a partir da classe *TransformCIMToPIM*, desenvolvida para ser usada no menu contextual, sendo necessário adicionar mais um comando no arquivo de configuração. A classe *TransformCIMToPIM* estende a classe abstrata *DefaultModuleCommandHandler* disponibilizada pela Modelio e sobreescreve os métodos *accept* e *actionPerformed*, além de definir dois métodos auxiliares, como *processDecisions* e *processPIMDiagram*.

O método *accept* desempenha o papel de verificar o escopo, determinando se a opção de transformação deve ser exibida para o elemento selecionado pelo usuário, como consequência, autoriza ou rejeita a transformação. Se a transformação for permitida, o método *actionPerformed* é acionado. A Figura 18 ilustra o código do método *actionPerformed* na classe *TransformCIMToPIM*. Esse método recupera a sessão atual do projeto, os estereótipos necessários do perfil Kuaba e o local para criação do diagrama PIM. Depois disso, percorre os elementos do diagrama buscando os que estão marcados com o estereótipo "Question", se for o caso, executa o método *processDecisions* para cada elemento

recuperado e, por fim, invoca o método *processPIMDiagram*.

Kuaba-Software-Engineering-tool - TransformCIMToPIM.java

```
1  @Override
2  public void actionPerformed(List<MObject> selectedElements, IModule module) {
3      IModelingSession session = module.getModuleContext().getModelingSession();
4
5      if (selectedElements.size() == 1 && selectedElements.get(0) instanceof Package) {
6          Package selectedPackage = (Package) selectedElements.get(0);
7          Package parent = (Package) selectedPackage.getOwner();
8
9          try (ITransaction transaction = session.createTransaction("Process Package Contents")) {
10             Stereotype questionElement = session.getMetamodelExtensions().getStereotype("KuabaModule", "Question", module.getModuleContext()
11                 .getModelioServices().getMetamodelService().getMetamodel().getMClass(Class.class));
12
13             Stereotype PIMPackage = session.getMetamodelExtensions()
14                 .getStereotype("KuabaModule", "PIMPackage", module.getModuleContext()
15                     .getModelioServices().getMetamodelService().getMetamodel().getMClass(Package.class));
16
17             Package targetPackage = session.getModel().createPackage("PIM Package", parent, PIMPackage);
18             List<Class> createdClasses = new ArrayList<>();
19
20             for (MObject element : selectedPackage.getCompositionChildren()) {
21                 if (element instanceof Class
22                 && questionElement != null && ((Class) element).isStereotyped(questionElement)) {
23                     createdClasses.addAll(processDecisions((Classifier) element, module, targetPackage, session));
24                 }
25             }
26             processPIMDiagram(module, targetPackage, createdClasses);
27             transaction.commit();
28         } catch (Exception e) {
29             module.getModuleContext().getLogService().error(e);
30         }
31     }
32 }
```

Figura 18: Código do método *actionPerformed* da classe *TransformCIMToPIM*

O método *processDecisions*, monstrado na Figura 19, recupera todas as associações relacionadas ao elemento *Question*, que são associações marcadas com o estereótipo *IsAddressedBy*. Para cada associação, ele verifica se a propriedade *Decision* contém o valor *Accepted*, certificando que a decisão para os elementos *Ideia*, aos quais apontam, foi aceita. Se isso for verdadeiro, uma nova classe é criada com o mesmo nome do elemento marcado com o estereótipo *Idea*,que está vinculada à associação. Quanto ao método *processPIMDiagram*, ele é responsável por criar o diagrama PIM e por inserir e organizar as classes criadas no diagrama.

Kuaba-Software-Engineering-tool - TransformCIMToPIM.java

```
1  private List<Class> processDecisions(Classifier element, IModule module, Package targetPackage, IModelingSession session) {
2
3      EList<AssociationEnd> associationEnds = element.getOwnedEnd();
4      List<Class> createdClasses = new ArrayList<>();
5
6      try (ITransaction t = session.createTransaction("Process Package Contents")) {
7          for (AssociationEnd associationEnd : associationEnds) {
8
9              if ("A".equals(associationEnd.getName())) {
10                 Class newClass = session.getModel().createClass(associationEnd.getTarget().getName(), targetPackage, null);
11                 createdClasses.add(newClass);
12             }
13         }
14
15         t.commit();
16     } catch (Exception e) {
17         module.getModuleContext().getLogService().error(e);
18     }
19     return createdClasses;
20 }
```

Figura 19: Código do método *processDecisions* da classe *TransformCIMToPIM*

6 Exemplo de uso

Para utilização do módulo, é necessário efetuar o *download* do arquivo "KuabaModule.jmdac", gerado na compilação do projeto e disponibilizado no github[2] do mesmo. Após o *download* da extensão é preciso adicionar no catálogo de módulos da Modelio e importá-lo em um projeto.

O módulo foi pensado e desenvolvido de forma a se destacar dos padrões da Modelio. Para a utilização, primeiro, o usuário deve, a partir do menu contextual, criar um pacote personalizado chamado "Kuaba Package", como mostra a Figura 20-a, onde os elementos Kuaba podem ser utilizados.

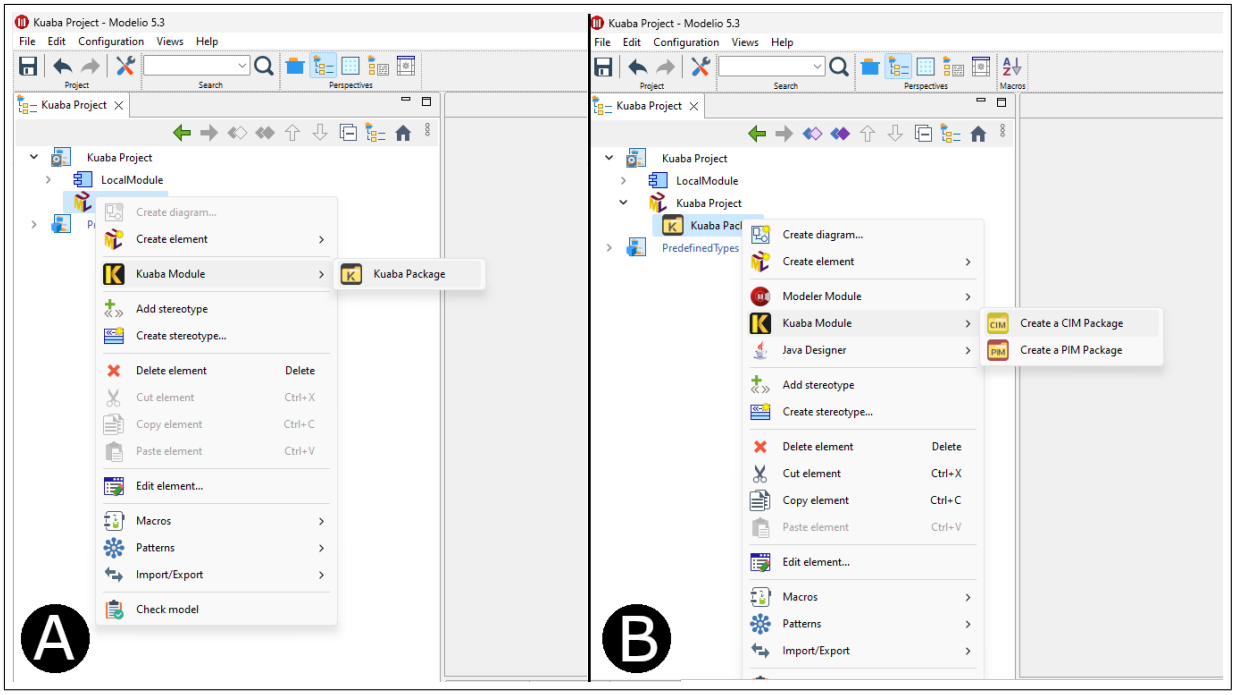


Figura 20: Criação dos pacotes *Kuaba Package*, CIM e PIM

A partir daí, existem mais dois tipos de pacote que podem ser criados manualmente: o *CIM Package*, para o modelo de discussão e o *PIM Package*, para o modelo PIM. O

[2]https://github.com/adrianamedeiros-uff/Kuaba-Software-Engineering-tool/releases

usuário tem liberdade para criar qualquer um dos pacotes e seus respectivos elementos. Porém o módulo foi projetado para ser iniciado com o modelo de discussão (CIM), que utiliza a ontologia Kuaba, e assim realizar as transformações automáticas: do modelo de discussão (CIM) para o PIM, depois do PIM para o PSM e, enfim, a geração de código a partir do PSM. A criação dos dois pacotes pode ser feita a partir do menu contextual, o usuário deve clicar com botão direito em *Kuaba Package*, selecionar a opção *Kuaba Module* e escolher o pacote que irá criar, como mostra a Figura 20-b.

O presente exemplo abordará a criação do diagrama Kuaba, que deve ser criado dentro do pacote CIM. Com o pacote criado, o usuário deve criar o diagrama relacionado ao pacote, ou seja, o diagrama Kuaba. O diagrama pode ser criado também a partir do menu contextual ou do *wizard* de criação. Para criar pelo menu contextual, basta clicar com botão direito no pacote CIM, selecionar a opção *Kuaba Module* e depois a opção *Create a Kuaba Diagram*. Já para a criação pelo *wizard*, o usuáro deve abrir o menu contextual para o pacote CIM e selecionar a opção *"create diagram..."*. A tela de criação com as especificações e diagramas separados por módulos será aberta. Após isso, o usuário deve selecionar o diagrama Kuaba dentro do Módulo Kuaba. A Figura 21-a mostra a criação através do menu contextual, enquanto a Figura 21-b mostra a criação através do *wizard* de criação.

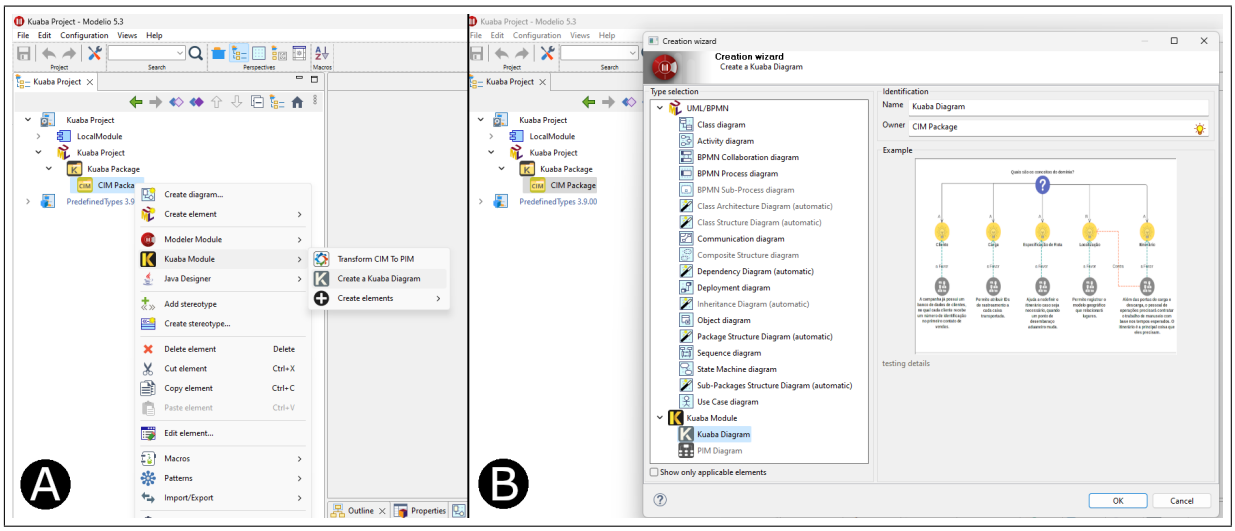


Figura 21: Criação de diagrama

Neste exemplo de uso é utilizado um domínio de gestão de biblioteca. Nesse domínio, os clientes podem realizar diferentes operações relacionadas à gestão de livros, empréstimos e devoluções. Os funcionários são responsáveis por gerenciar as operações feitas pelos clientes. Os livros podem ser emprestados e devolvidos por um cliente, além de possuir

uma editora e uma ou mais categorias. Os exemplares de livro são instâncias individuais de cada livro que possui estoque maior que um.

No modelo de discussão os conceitos do domínio são representados como ideias (ícones de lâmpadas) que respondem à questão principal (ícone de interrogação). Nesse exemplo, os conceitos considerados são Cliente, Funcionário, Livro, Cópia de Livro, Empréstimo, Devolução, Categoria e Editora. Com isso, o primeiro passo é incluir um elemento Questão, que será a raiz do diagrama e alterar o nome na aba de propriedades do Módulo Kuaba para algo como "Quais são os elementos do Modelo?"(Figura 22).

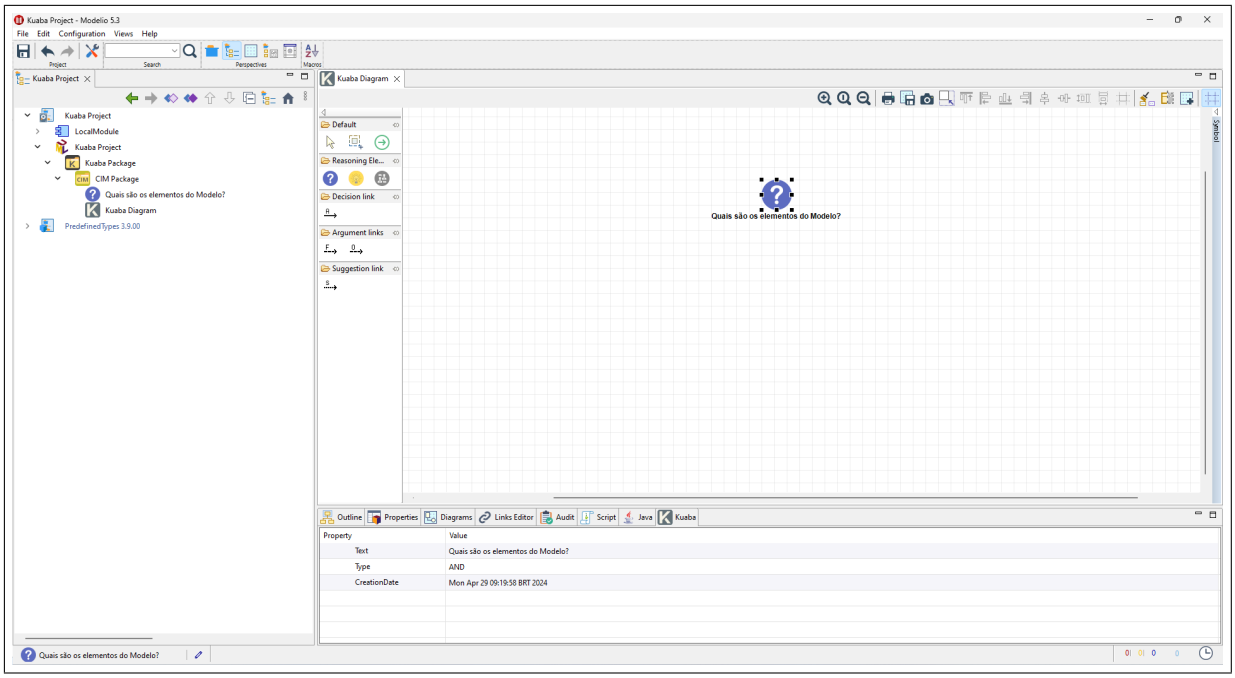


Figura 22: Criação do elemento Questão

Agora é necessário incluir as ideias que respondem a esta questão e os argumentos a favor ou contra essas ideias. Para isso, é preciso adicionar no diagrama as ideias e os argumentos (ícone de diálogo), alterando o campo *text* da aba de propriedades. para as ideias esse campo se refere ao nome do elemento, enquanto para os argumentos o campo se refere ao argumento em si. Além disso, os argumentos possuem um campo chamado *Display Argument*, utilizado para aspectos visuais e economia de espaço, definindo se o argumento inteiro será exibido ou apenas o texto *"argument"*. Com os argumentos definidos, o usuário deve criar um *link* a favor (ícone de seta com a letra F) ou contra (ícone de seta com a letra O) partindo do argumento para a ideia. Os passos descritos são mostrados na Figura 23 e na Figura 24.

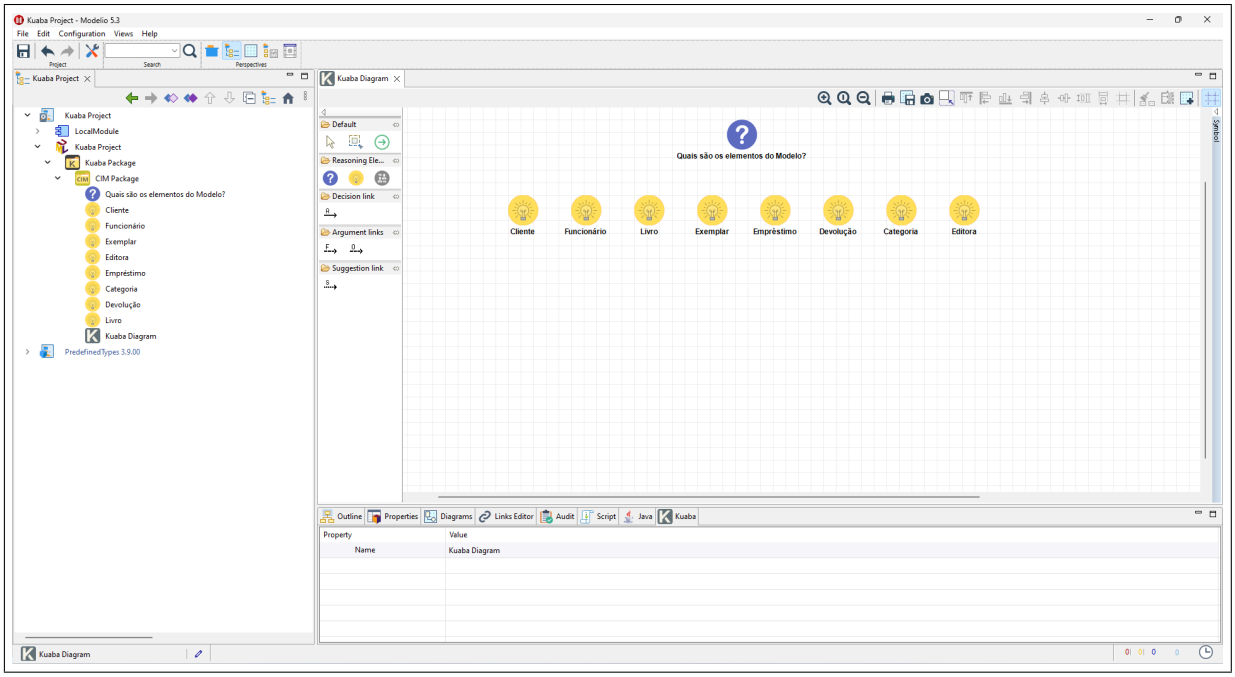


Figura 23: Criação dos elementos Ideia

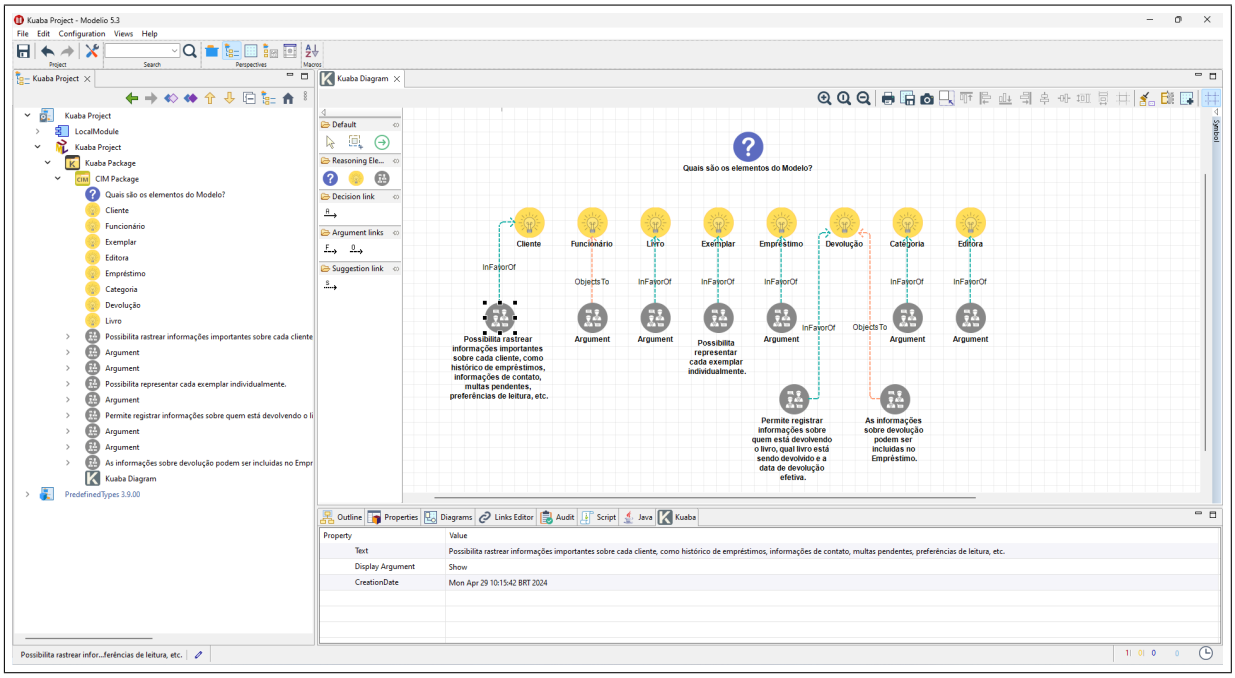


Figura 24: Criação dos elementos Argumento

Após as argumentações as decisões serão tomadas, ponderando possíveis conflitos e justificando o porque da decisão. Para definir as decisões, o *link* de decisão (ícone de seta com a letra A) deve ser utilizado para ligar a questão às ideias e através da aba de propriedades, deve definir no campo decisão se foi aceita ou não e no campo justificativa

o motivo da decisão tomada. A Figura 25 mostra como esses elementos são exibidos, a decisão aceita possui uma seta continua com a letra A (*Accepted*), enquanto a rejeitada possui a letra R (*Rejected*).

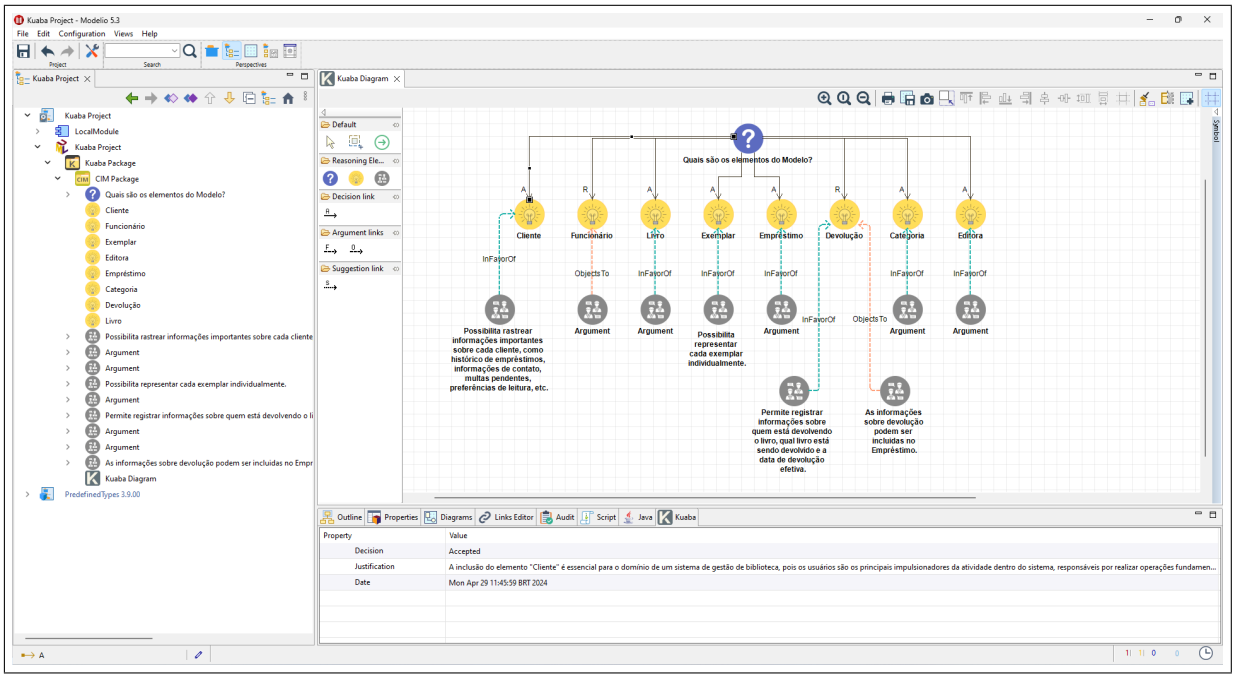


Figura 25: Criação dos elementos de Decisão

Com o diagrama Kuaba finalizado, é hora de realizar a transformação de CIM para PIM, clicando com o botão direito no pacote CIM, que contém o diagrama, selecionando *Kuaba Module* e depois a opção *transform CIM to PIM*, como mostra a Figura26. Assim, a transformação é executada. Como resultado um pacote PSM é criado, contendo um diagrama com as classes relacionadas às ideias aceitas, como ilustra a Figura 27.

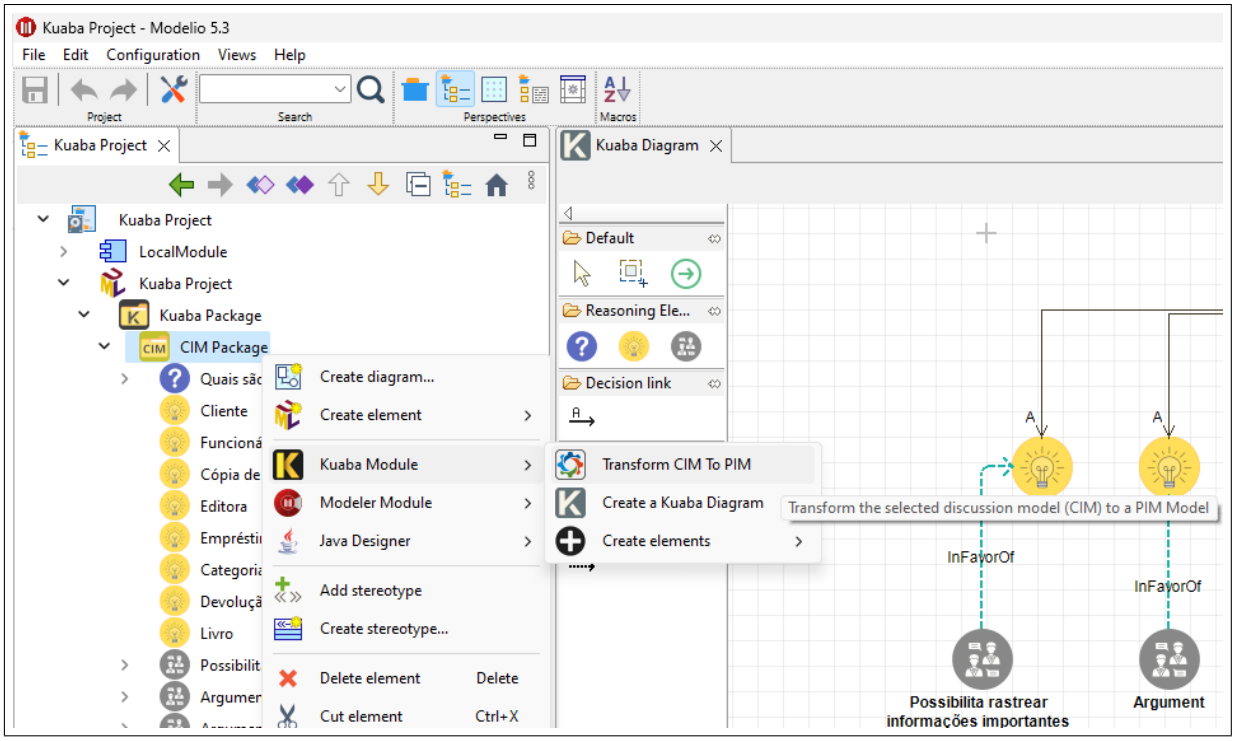


Figura 26: Execução da transformação

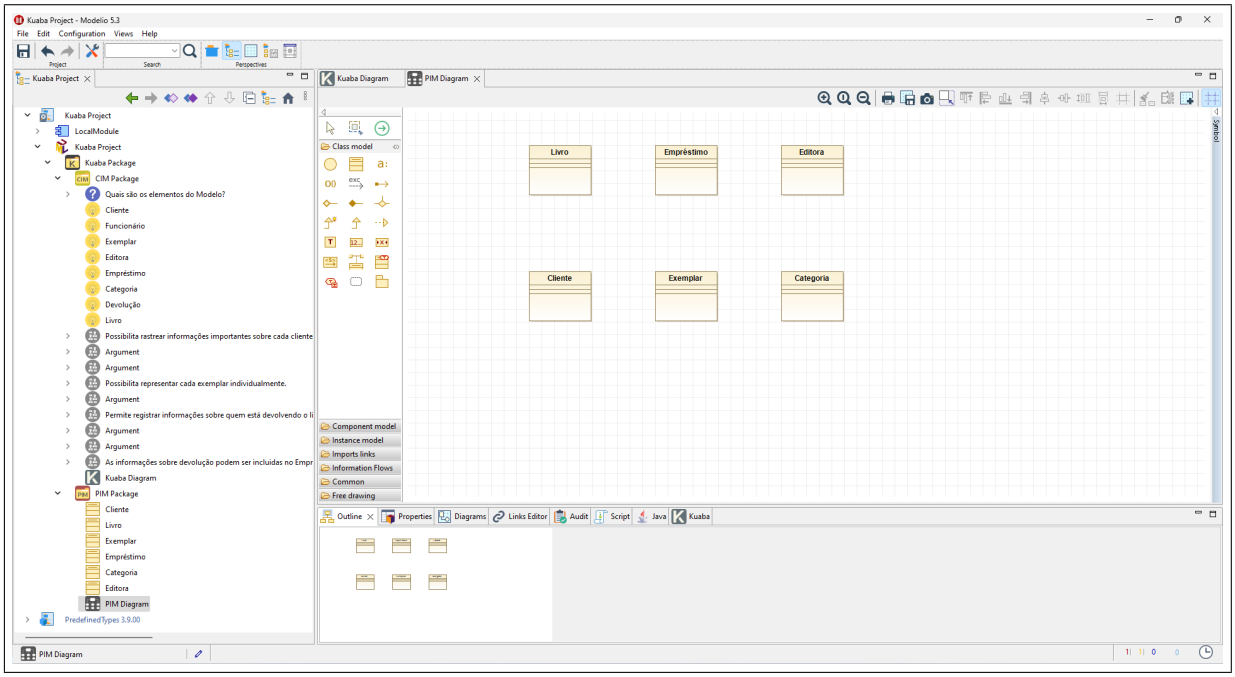


Figura 27: PIM após transformação

O próximo passo é construir o PIM, inserindo os atributos, as operações e as relações entre as classes, como mostra a Figura 28. Neste modelo, é possível marcar os elementos com os estereótipos do perfil DDD e realizar as transformações PIM para PSM e PSM

para código.

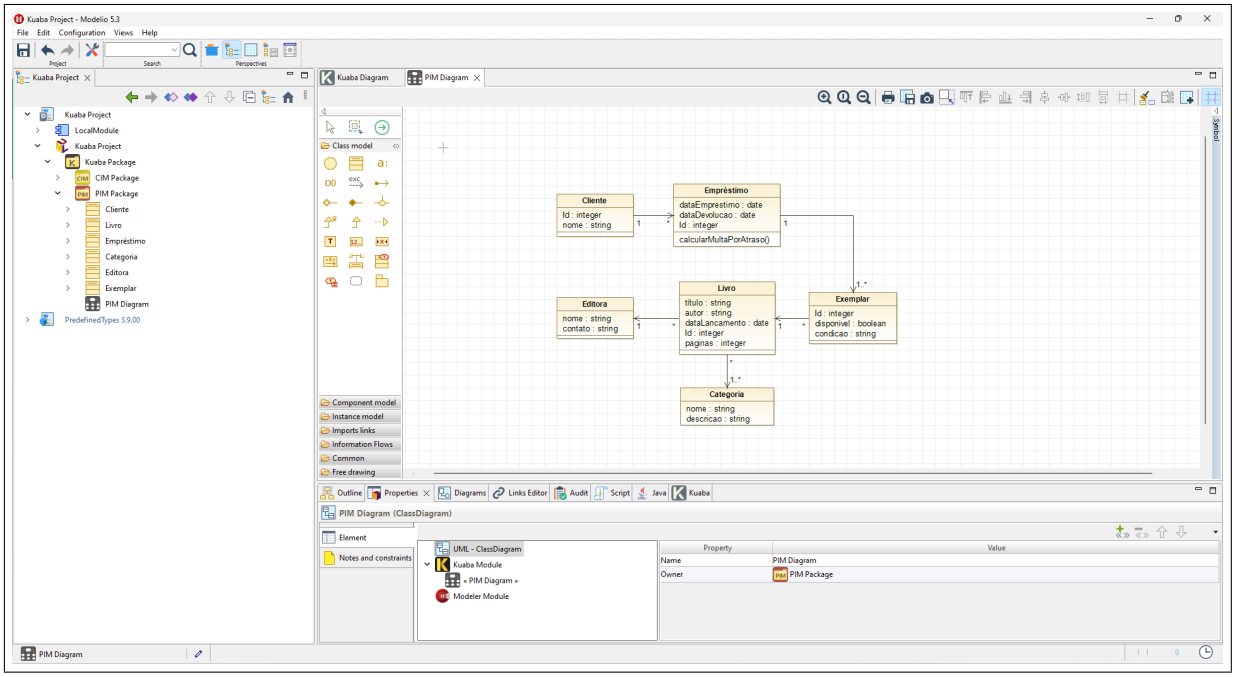


Figura 28: PIM finalizado pelo usuário

7 Considerações finais

Este trabalho apresentou a implementação de uma extensão para a ferramenta modelio, utilizando a abordagem Kuaba em conjunto com MDE. Foi desenvolvido um módulo que fornece toda a estrutura necessária para os processos MDA, abrangendo pacotes e diagramas CIM (Kuaba), PIM e PSM. O módulo dá suporte para criação do modelo de discussão (CIM), a partir do perfil Kuaba com interface gráfica e funcionalidades personalizadas, além da execução da transformação entre modelos de forma intuitiva. No escopo deste trabalho foi feita apenas a transformação do CIM para o PIM, onde o PIM pode ser gerado a partir das decisões registradas no CIM. Depois foi feita a integração com a outra parte do projeto.

Algumas melhorias podem ser realizadas em um trabalho futuro, como a criação de outra aba para o PIM, para ajudar a definir com maior facilidade os estereótipos DDD, que são necessários para realizar as transformações de PIM para PSM. Outra melhoria é implementar a geração de um relatório com todas as escolhas e informações definidas no modelo de discussão pelo usuário, o que facilitaria a documentação.

REFERÊNCIAS

BUCCHIARONE, Antonio *et al.* Grand challenges in model-driven engineering: an analysis of the state of the research. **Software and Systems Modeling**, Springer, v. 19, p. 5–13, 2020.

CUNHA, Luiz Eduardo. **Transformações de modelos de domínio utilizando a arquitetura MDA para apoiar a geração automática de código**. 2023. Trabalho de Conclusão de Curso – Universidade Federal Fluminense.

DAVID S. FRANKEL, Paul Harmon *et al.* **The Zachman Framework and the OMG's Model Driven Architecture**. 2003. Disponível em: https://www.omg.org/mda/mda_files/09-03-WP_Mapping_MDA_to_Zachman_Framework1.pdf. Acesso em: 28 abr. 2024.

LEE, J. Design rationale systems: understanding the issues. **IEEE Expert**, v. 12, n. 3, p. 78–85, 1997. DOI: 10.1109/64.592267.

MEDEIROS, Adriana. **Kuaba: Uma Abordagem para Representação de Design Rationale para o Reuso de Designs baseados em Modelo**. 2006. Tese de Doutorado – Pontifícia Universidade Católica do Rio de Janeiro. DOI: https://doi.org/10.17771/PUCRio.acad.8935.

MEDEIROS, Adriana. Pesquisando Soluções para Desafios em MDE com Kuaba, p. 11–15, out. 2020. DOI: 10.5753/mssis.2020.12490.

MODELIO. **MODELIO**. Modelio Open Source. Disponível em: https://github.com/ModelioOpenSource/Modelio. Acesso em: 22 abr. 2024.

MODELIOSOFT. **Extensibility and Customization Features**: Every Modelio product can be adapted to your environment to raise quality and productivity. Modeliosoft. Disponível em: https://www.modeliosoft.com/en/features/extensibility-and-customization.html. Acesso em: 22 abr. 2024.

MODELIOSOFT. **MDA**: MDA is Model-Driven Architecture technology. Modeliosoft. Disponível em: https://www.modeliosoft.com/en/technologies/mda.html. Acesso em: 22 abr. 2024.

OMG. **Model Driven Architecture (MDA)**. OMG. 2001. Disponível em: https://www.omg.org/cgi-bin/doc?ormsc/01-07-01.pdf. Acesso em: 30 abr. 2024.

OMG. **OMG Model driven architecture (MDA)**: MDA Guide rev. 2.0. OMG. 2014. Disponível em: https://www.omg.org/cgi-bin/doc?ormsc/14-06-01.pdf. Acesso em: 30 abr. 2024.

RADEMACHER, Florian; SACHWEH, Sabine; ZÜNDORF, Albert. Towards a UML profile for domain-driven design of microservice architectures. *In:* SPRINGER. SOFTWARE Engineering and Formal Methods: SEFM 2017 Collocated Workshops: DataMod, FAACS, MSE, CoSim-CPS, and FOCLASA, Trento, Italy, September 4-5, 2017, Revised Selected Papers 15. [*S. l.: s. n.*], 2018. p. 230–245.